



武汉大学
Wuhan University

第四章 组合逻辑电路

4.4 常用组合逻辑电路

4.5 竞争与险象

武汉大学计算机学院 2025-2026第二学期



主要内容

Main Content

数据选择器

数据分配器

数值比较器

竞争和险象

4.4.4 数据选择器

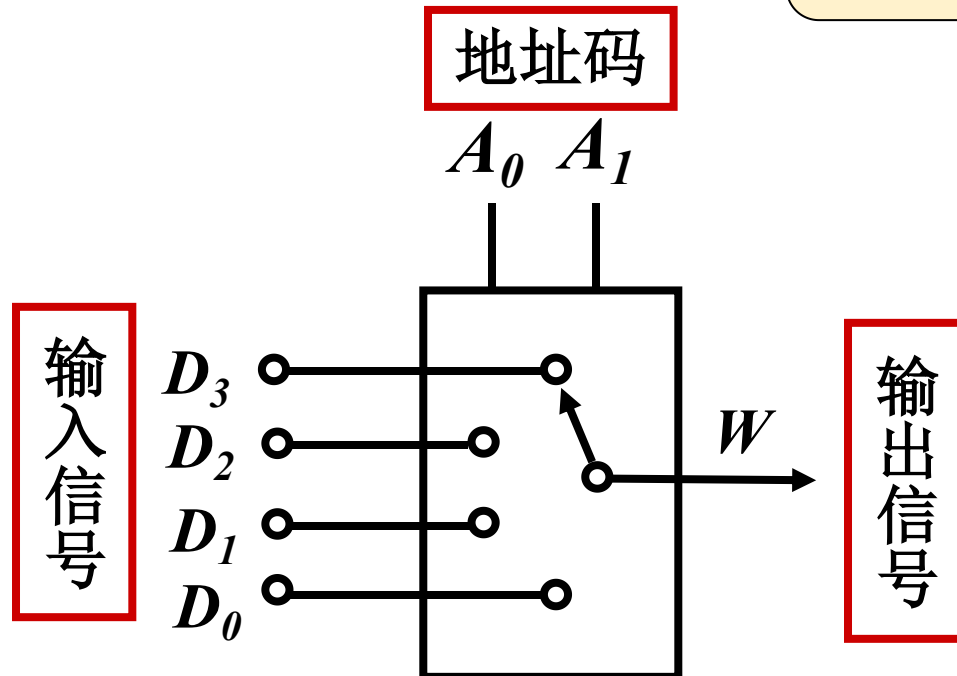
定义：能从多个数据信号中选择~~一个~~数据信号传送到输出端的电路。

每一路输入数据有一个编号

输入： 2^n 路数据和 n 位地址

输出：1位数据

地址码（控制信号）：控制选择哪一路数据信号。



数据选择器类似一个多掷开关。输入数据中编号与地址码相同的那一路数据被选中输出。

4.4.4 数据选择器

4选1数据选择器

输入数据: D_3, D_2, D_1, D_0
控制地址: A_1, A_0
输出数据: F
功能: 根据输入地址, 选择输入数据中的一个, 送到输出端

真值表

A_1	A_0	F
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$F = \overline{A_1} \overline{A_0} D_0 + \overline{A_1} A_0 D_1 + A_1 \overline{A_0} D_2 + A_1 A_0 D_3$$

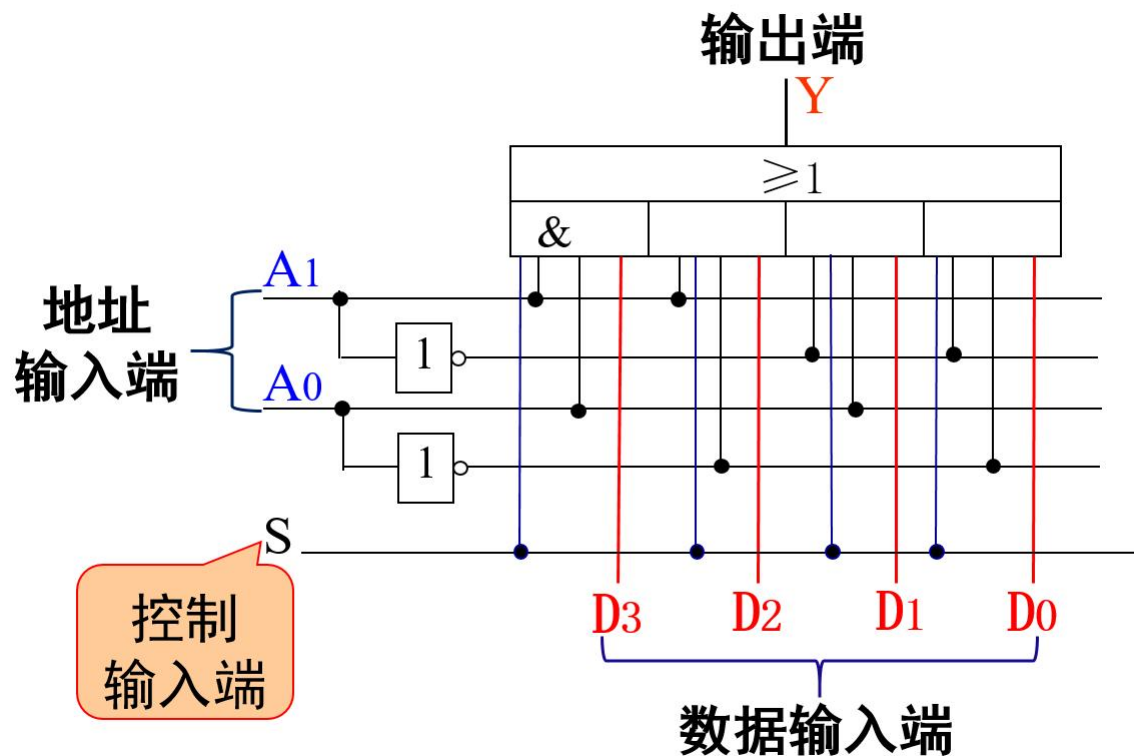
表达式:
$$= m_0 \cdot D_0 + m_1 \cdot D_1 + m_2 \cdot D_2 + m_3 \cdot D_3$$

$$= \sum_{i=0}^{2^n-1} m_i \cdot D_i = \sum_{i=0}^3 m_i \cdot D_i$$

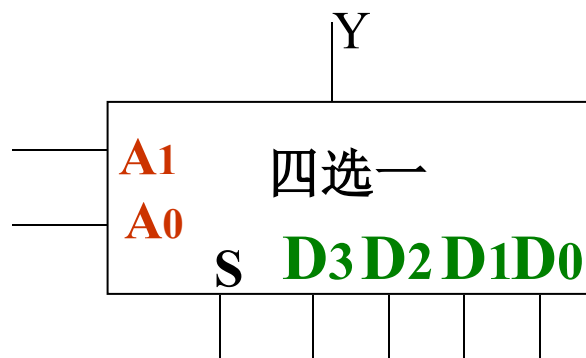
4.4.4 数据选择器

4选1数据选择器

$$F = \sum_{i=0}^3 m_i \cdot D_i$$



输入			输出
S	A_1	A_0	Y
0	×	×	0
1	0	0	D_0
1	0	1	D_1
1	1	0	D_2
1	1	1	D_3



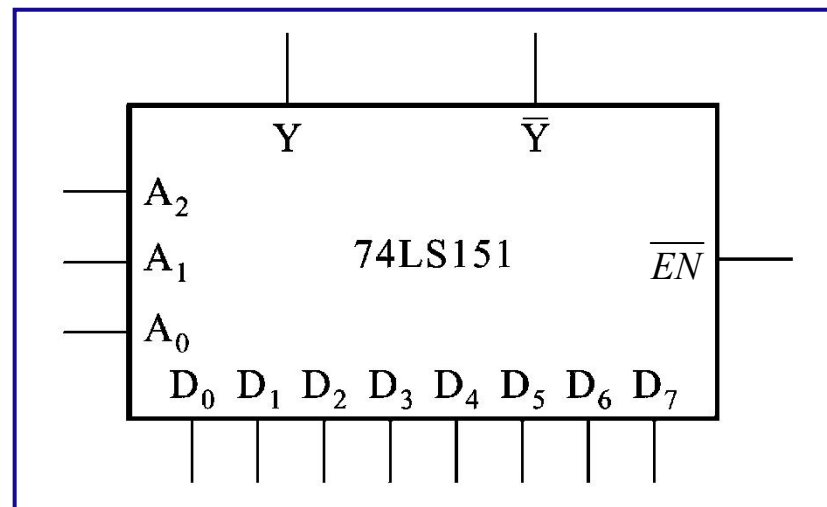
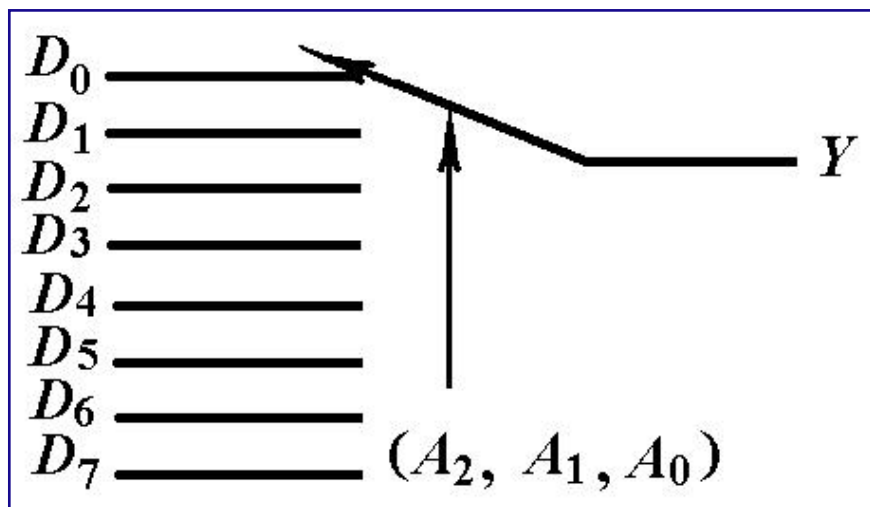
4.4.4 数据选择器

4选1数据选择器

```
module mux4(iD,iS,oQ);  
    input [3:0] iD;           //数据输入信号  
    input [1:0] iS;           //数据选择控制信号  
    output reg oQ;            //输出信号  
  
    always@(iD,iS)  
        case(iS)  
            2'b00: oQ=iD[0];  
            2'b01: oQ=iD[1];  
            2'b10: oQ=iD[2];  
            2'b11: oQ=iD[3];  
        endcase  
endmodule
```

4.4.4 数据选择器

8选1数据选择器74LS151



74LS151的逻辑符号

三个地址输入端： A_2 , A_1 , A_0

八个数据输入端： $D_0 \sim D_7$

两个互补输出的数据输出端： Y , $\bar{Y}$,

一个控制输入端： $\overline{EN}$

4.4.4 数据选择器

8选1数据选择器74LS151

输 入				输 出	
$\overline{EN}$	A_2	A_1	A_0	Y	$\overline{Y}$
1	×	×	×	0	1
0	0	0	0	D_0	$\overline{D_0}$
0	0	0	1	D_1	$\overline{D_1}$
0	0	1	0	D_2	$\overline{D_2}$
0	0	1	1	D_3	$\overline{D_3}$
0	1	0	0	D_4	$\overline{D_4}$
0	1	0	1	D_5	$\overline{D_5}$
0	1	1	0	D_6	$\overline{D_6}$
0	1	1	1	D_7	$\overline{D_7}$

使能时，
输出和输入的逻辑关系：

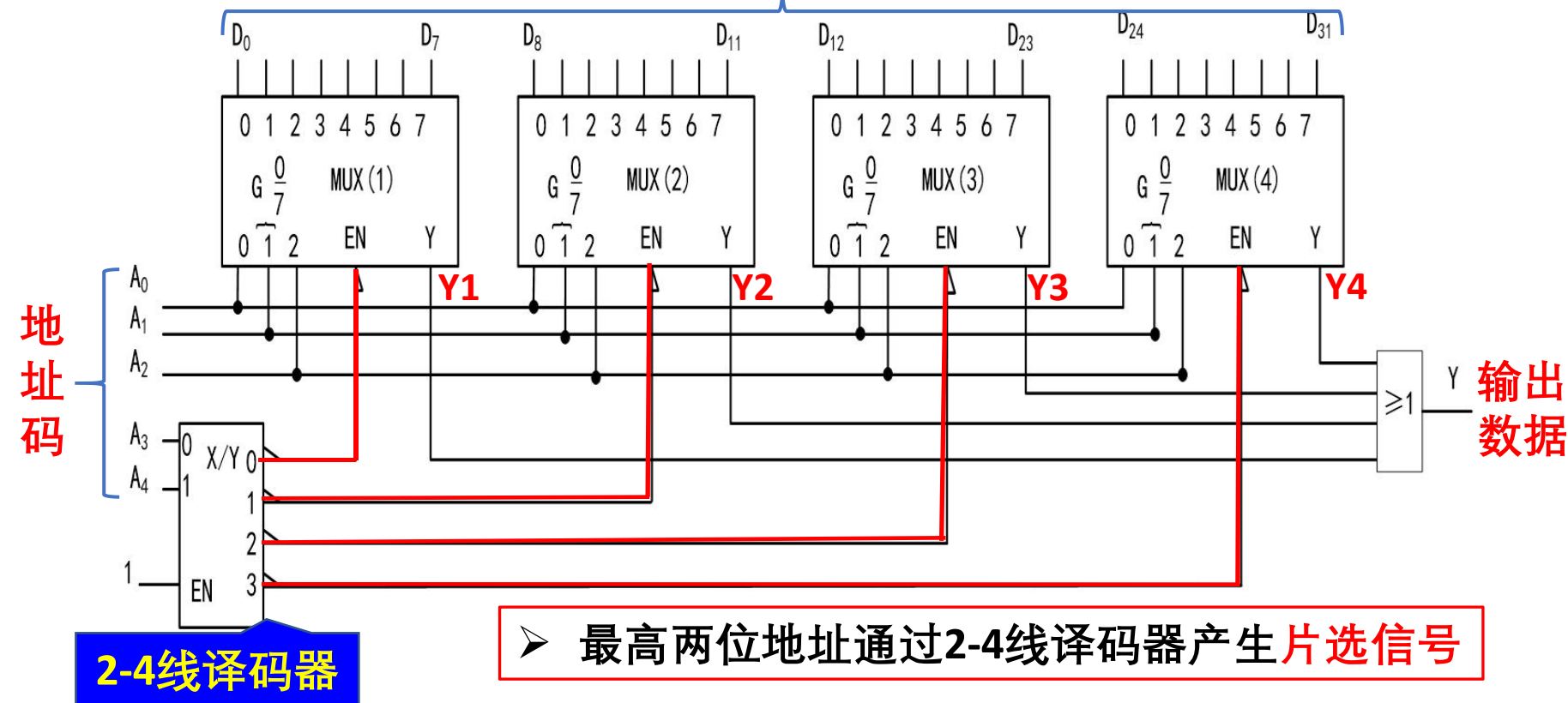
$$\begin{aligned}
 Y &= \overline{A_2} \overline{A_1} \overline{A_0} D_0 + \overline{A_2} \overline{A_1} A_0 D_1 \\
 &\quad + \overline{A_2} A_1 \overline{A_0} D_2 + \overline{A_2} A_1 A_0 D_3 \\
 &\quad + A_2 \overline{A_1} \overline{A_0} D_4 + A_2 \overline{A_1} A_0 D_5 \\
 &\quad + A_2 A_1 \overline{A_0} D_6 + A_2 A_1 A_0 D_7 \\
 &= \sum m_i D_i
 \end{aligned}$$

地址变量
构成的最
小项

4.4.4 数据选择器

用4片8选1数据选择器构成32选1数据选择器（方法一）

输入数据

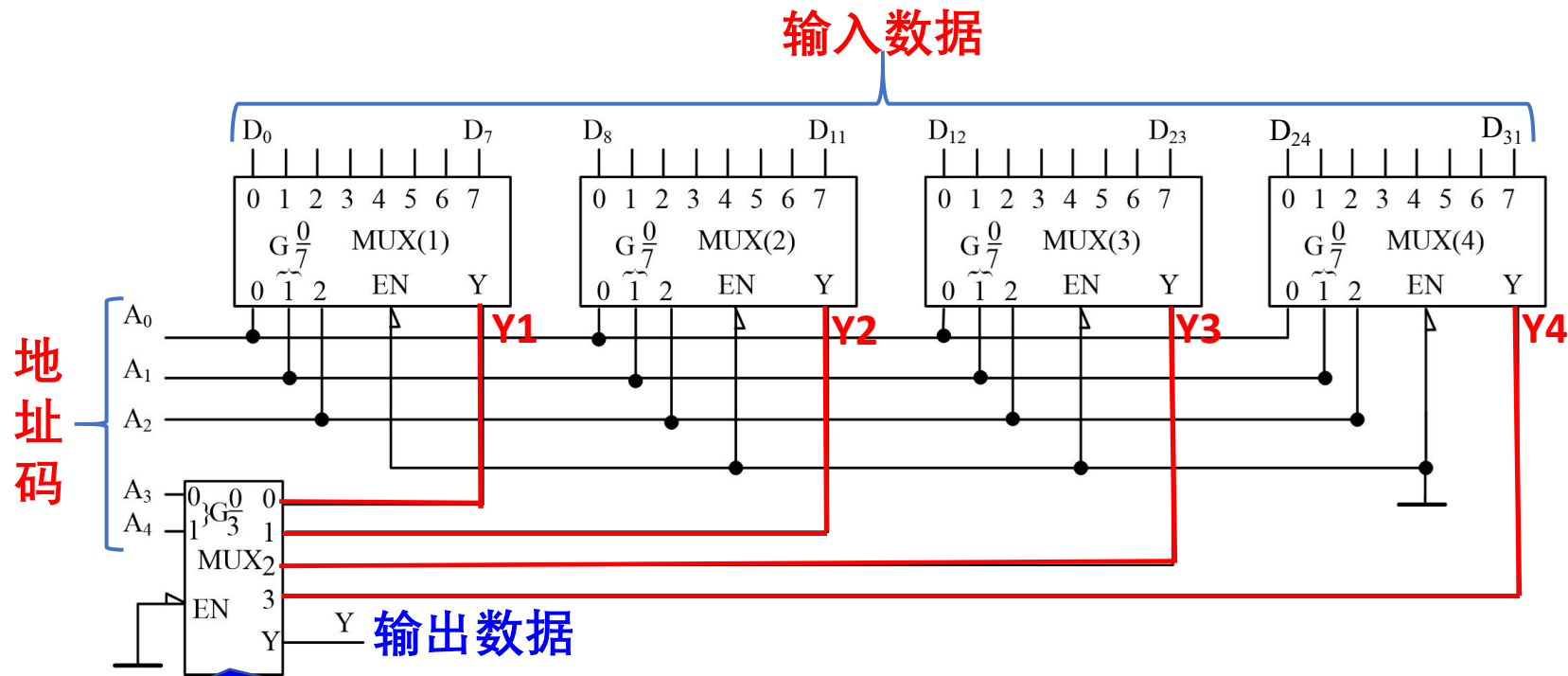


$A_4 A_3 = \begin{cases} 00 & \text{MUX(1)工作} \\ 01 & \text{MUX(2)工作} \\ 10 & \text{MUX(3)工作} \\ 11 & \text{MUX(4)工作} \end{cases}$

地址 $A_4 A_3 A_2 A_1 A_0$ 取值**0000-00111**: 输出信号来自 **D_0-D_7**
 地址 $A_4 A_3 A_2 A_1 A_0$ 取值**01000-01111**: 输出信号来自 **D_8-D_{15}**
 地址 $A_4 A_3 A_2 A_1 A_0$ 取值**10000-10111**: 输出信号来自 **$D_{16}-D_{23}$**
 地址 $A_4 A_3 A_2 A_1 A_0$ 取值**11000-11111**: 输出信号来自 **$D_{24}-D_{31}$**

4.4.4 数据选择器

用4片8选1数据选择器构成32选1数据选择器（方法二）



4选1数据选择器

➤ 最高两位地址通过4选1数据选择器，从4片8选1数据选择器的输出信号中选择一个，作为整个电路的输出。

$A_4 A_3$

- 00: $Y=Y_1$
- 01: $Y=Y_2$
- 10: $Y=Y_3$
- 11: $Y=Y_4$

地址 $A_4 A_3 A_2 A_1 A_0$ 取值 **0000-0011**: 输出信号来自 D_0-D_7
 地址 $A_4 A_3 A_2 A_1 A_0$ 取值 **0100-0111**: 输出信号来自 D_8-D_{15}
 地址 $A_4 A_3 A_2 A_1 A_0$ 取值 **1000-1011**: 输出信号来自 $D_{16}-D_{23}$
 地址 $A_4 A_3 A_2 A_1 A_0$ 取值 **1100-1111**: 输出信号来自 $D_{24}-D_{31}$

主要内容

Main Content

数据选择器

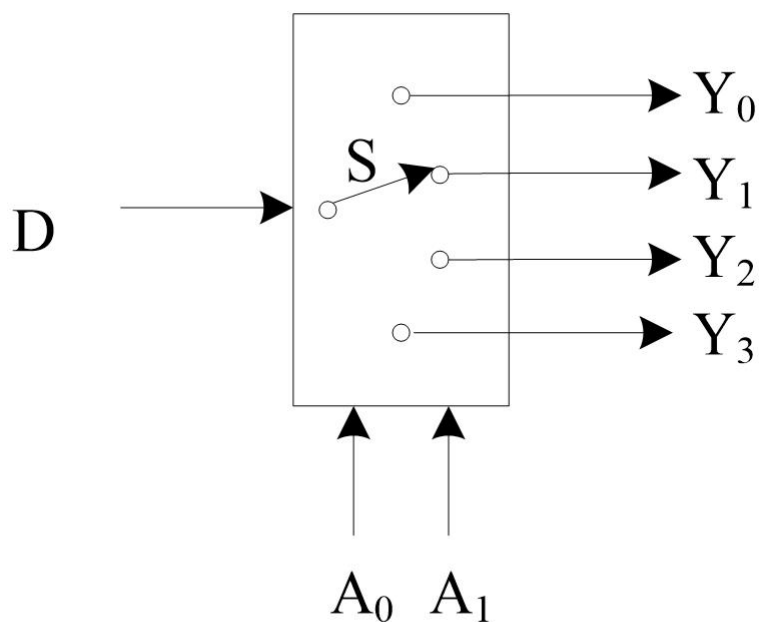
数据分配器

数值比较器

竞争和险象

4.4.5 数据分配器

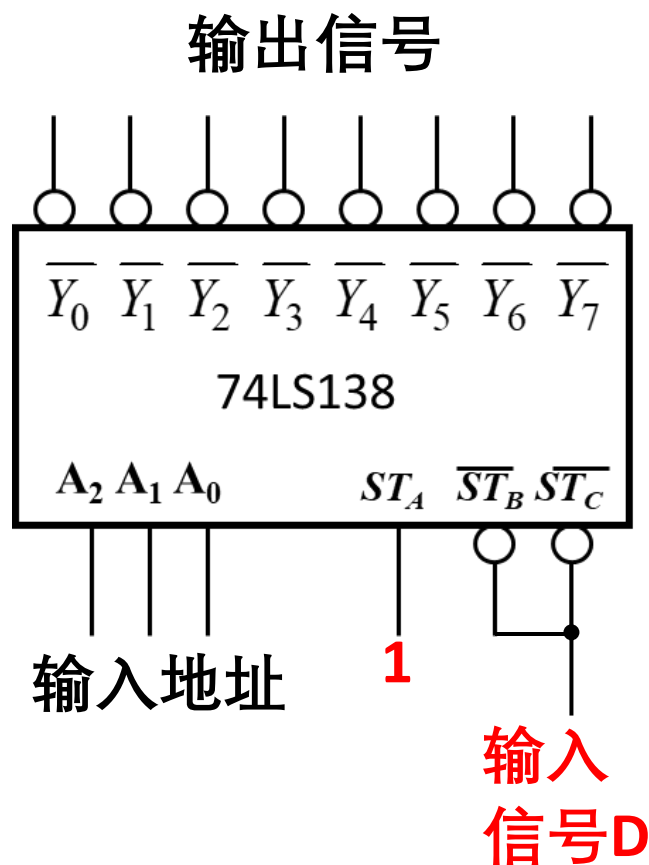
在数据传送中，有时需要将某一路数据分配到不同的数据通道上，实现这种功能的电路称为数据分配器，也称**多路分配器**。



市场上并没有专用的数据分配器器件，实际使用中，通常用**译码器**来实现数据分配的功能。

4.4.5 数据分配器

用3-8译码器实现8路数据分配器



假设输入地址： $A_2A_1A_0=010$

- 若输入信号 $D=0$ ，则输出端
 $\overline{Y_2} = 0 = D$
- 如输入信号 $D=1$ ，则芯片禁止，所有输出端都为1，所以此时
 $\overline{Y_2} = 1 = D$

结论：

- 输入地址指定的输出端信号等于输入信号
- 改变输入地址就能将输入信号D分配到不同输出端

4.4.5 数据分配器

module dmux_8(iEN, iS, iD, oY); //8路数据分配器模块定义

```
module dmux_8(iEN, iS, iD, oY);  
    input iEN;           //使能控制信号  
    input iD;           //数据输入信号  
    input [2:0] iS;      //地址信号  
    output reg [7:0] oY; //数据输出信号  
    always@(iD, iEN, iS)  
    begin  
        oY=8'b11111111;  
        if(iEN)  
            case(iS)  
                3'b000:oY[0]=iD;  
                3'b001:oY[1]=iD;  
                3'b010:oY[2]=iD;  
                3'b011:oY[3]=iD;
```

```
                3'b100:oY[4]=iD;  
                3'b101:oY[5]=iD;  
                3'b110:oY[6]=iD;  
                3'b111:oY[7]=iD;  
            endcase  
        end  
    Endmodule
```

主要内容

Main Content

数据选择器

数据分配器

数值比较器

竞争和险象

4.4.6 数值比较器

比较器：能完成两个数字大小及相同比较的电路

功 能：大小比较、相同比较

真值表

1. 一位数值比较器

输入：数据A、B

输出： $F_{A>B}$ 、 $F_{A=B}$ 、 $F_{A<B}$

A	B	$F_{A>B}$	$F_{A=B}$	$F_{A<B}$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

4.4.6 数值比较器

1. 一位数值比较器

真值表

A	B	$F_{A>B}$	$F_{A=B}$	$F_{A<B}$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

表达式

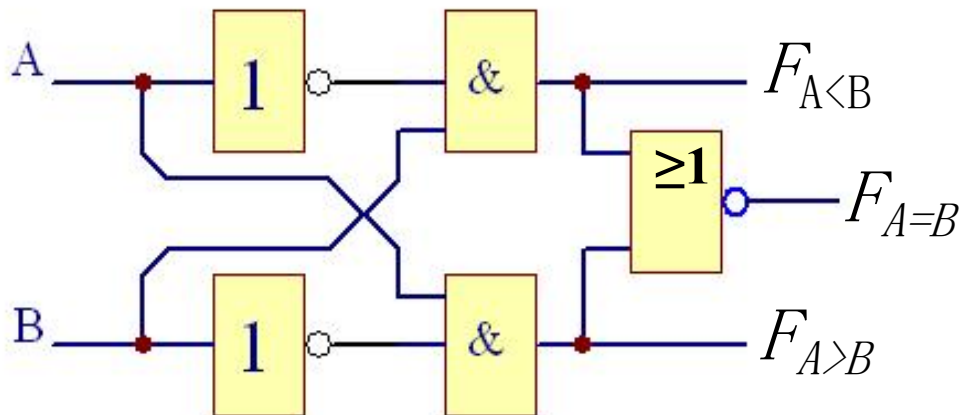
$$F_{A>B} = m_2 = A\bar{B}$$

$$F_{A=B} = m_0 + m_3 = \bar{A}\bar{B} + AB$$

$$= \overline{A\bar{B} + \bar{A}B} = \overline{A \oplus B}$$

$$F_{A<B} = m_1 = \bar{A}B$$

逻辑图



4.4.6 数值比较器

2. 二位数值比较器

数 值 输 入		级 联 输 入			输 出				
$A1$	$B1$	$A0$	$B0$	$I_{A>B}$	$I_{A<B}$	$I_{A=B}$	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
$A1 > B1$		×	×	×	×	×	1	0	0
$A1 < B1$		×	×	×	×	×	0	1	0
$A1 = B1$	$A0 > B0$			×	×	×	1	0	0
$A1 = B1$	$A0 < B0$			×	×	×	0	1	0
$A1 = B1$	$A0 = B0$			1	0	0	1	0	0
$A1 = B1$	$A0 = B0$			0	1	0	0	1	0
$A1 = B1$	$A0 = B0$			0	0	1	0	0	1

$$F_{A>B} = (A_1 > B_1) + (A_1 = B_1) \cdot (A_0 > B_0) + (A_1 = B_1) \cdot (A_0 = B_0) \cdot I_{A>B}$$

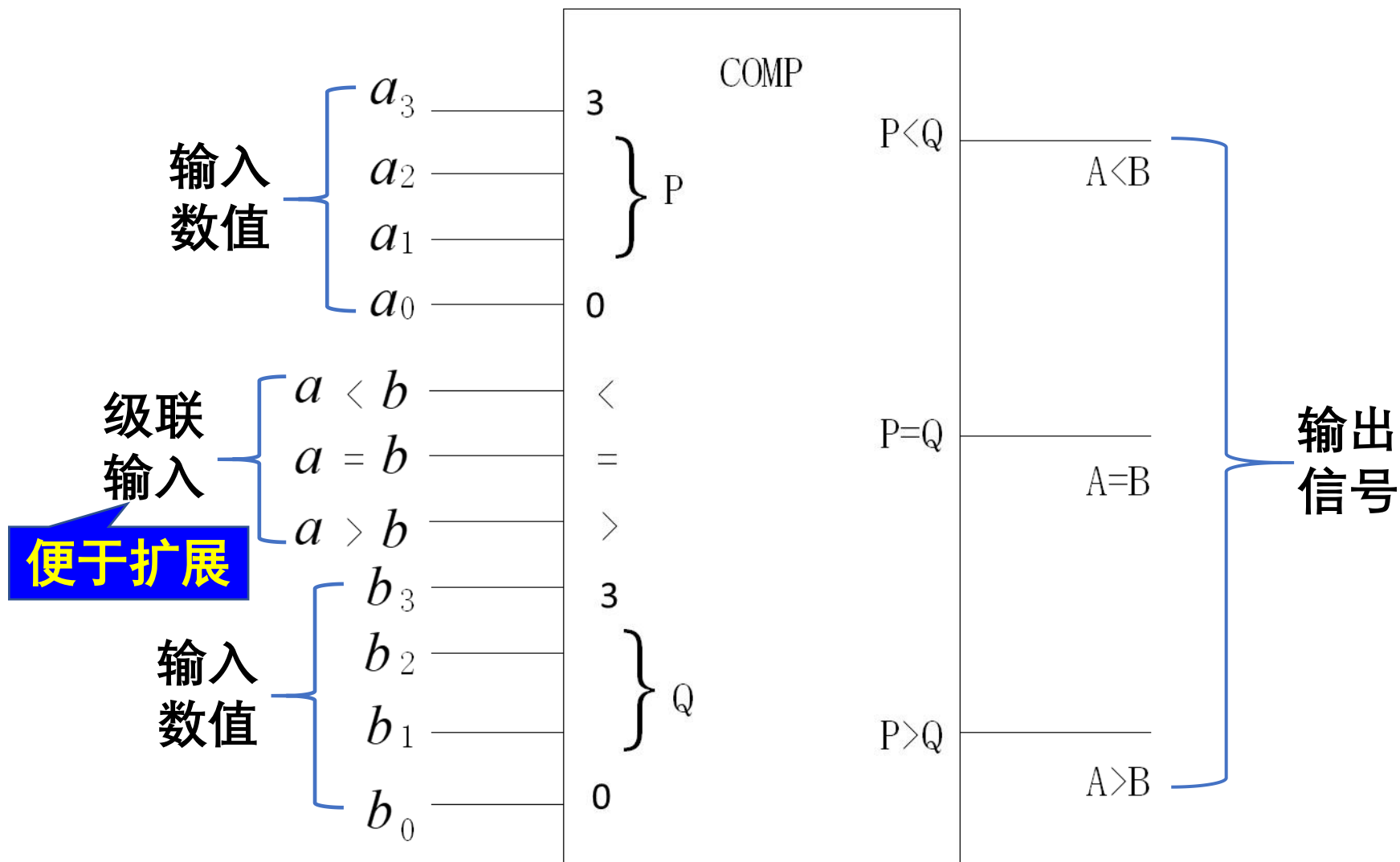
$$F_{A<B} = (A_1 < B_1) + (A_1 = B_1) \cdot (A_0 < B_0) + (A_1 = B_1) \cdot (A_0 = B_0) \cdot I_{A<B}$$

$$F_{A=B} = (A_1 = B_1) \cdot (A_0 = B_0) \cdot I_{A=B}$$

如果要比较两个多位二进制数A和B的大小,必须从**高**向**低**逐位进行比较。

4.4.6 数值比较器

3. 四位数值比较器-74LS85

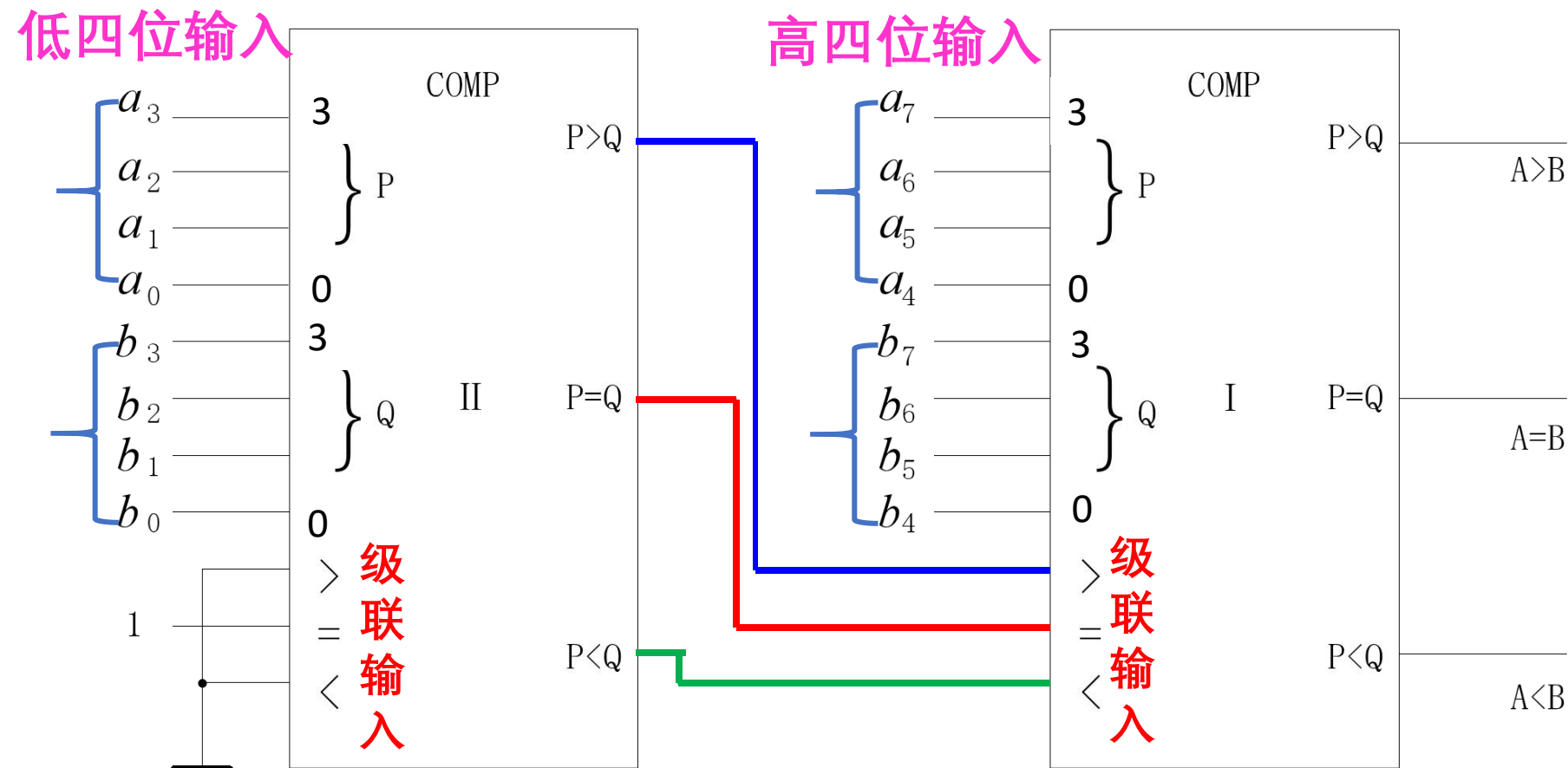


(b)

4.4.6 数值比较器

3. 四位数值比较器-74LS85

用2片4位数值比较器74LS85扩展成8位数值比较器



串行级联构成的八位比较器

4.4.6 数值比较器

4. 多位数值比较器的Verilog HDL设计

```
module samp4_4_10(a, b, great, less, equ); //调用比较器模块的顶层模块
    input [7:0] a, b;
    output great, less, equ;
    compare_n #(8) u1(.A(a), .B(b), .AGB(great), .ALB(less), .AEB(equ));
    //调用8位比较器模块
endmodule

module compare_n(A, B, AGB, ALB, AEB); //比较器模块
    input [n-1:0] A, B;
    output reg AGB, ALB, AEB;
    parameter n=4;
    always@(A, B)
        begin
            AGB=0;
            ALB=0;
            AEB=0;
            if (A>B) GB=1;
            else if(A==B) AEB=1;
            else ALB=1;
        end
endmodule
```

主要内容

Main Content

数据选择器

数据分配器

数值比较器

竞争和险象

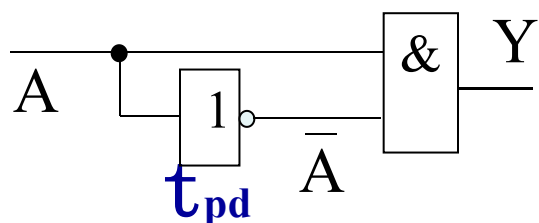
4.5.1 基本概念

理想情况：

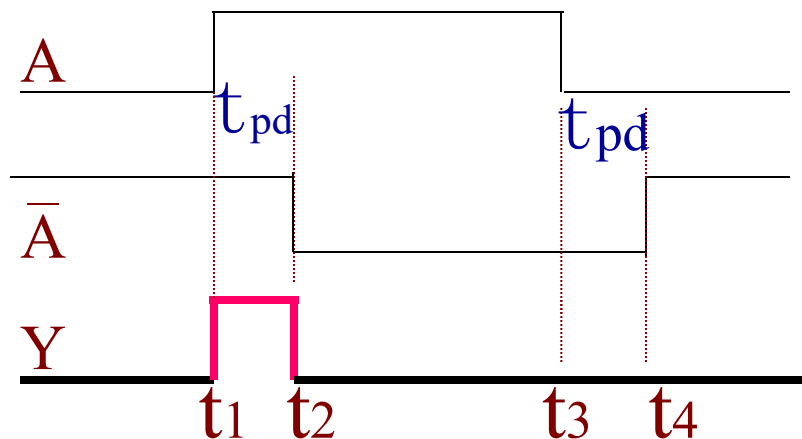
- 逻辑门连线无延迟
- 多个信号同时瞬间变化

实际情况：

- 信号变化：过渡时间
- 信号通过逻辑门：响应时间
- 多个信号变化：有先有后



$$Y = A \bar{A} \quad \left\{ \begin{array}{l} \text{静态时: } Y \equiv 0 \\ \text{动态且 } t_{pd} \neq 0 \text{ 时: } Y = ? \end{array} \right.$$



结果，在 $t_1 \sim t_2$ 时间内，电路输出端产生了 $Y=1$ 的尖峰脉冲，

它不符合静态下 $Y = A\bar{A}$ 恒为0的逻辑关系。

4.5.1 基本概念

竞争：信号经不同路径到达某一点时，所用的时间不同，这个**时间差**称为**竞争**

险象：由竞争引起电路输出发生**瞬间错误**的现象，表现为输出端出现了原设计中没有的**窄脉冲**（毛刺），称为**险象**

- 一般来说，时延对数字系统是有害的，它会降低系统的工作速度，还会产生竞争冒险现象
- 竞争和险象是对电路的，而不是针对函数的

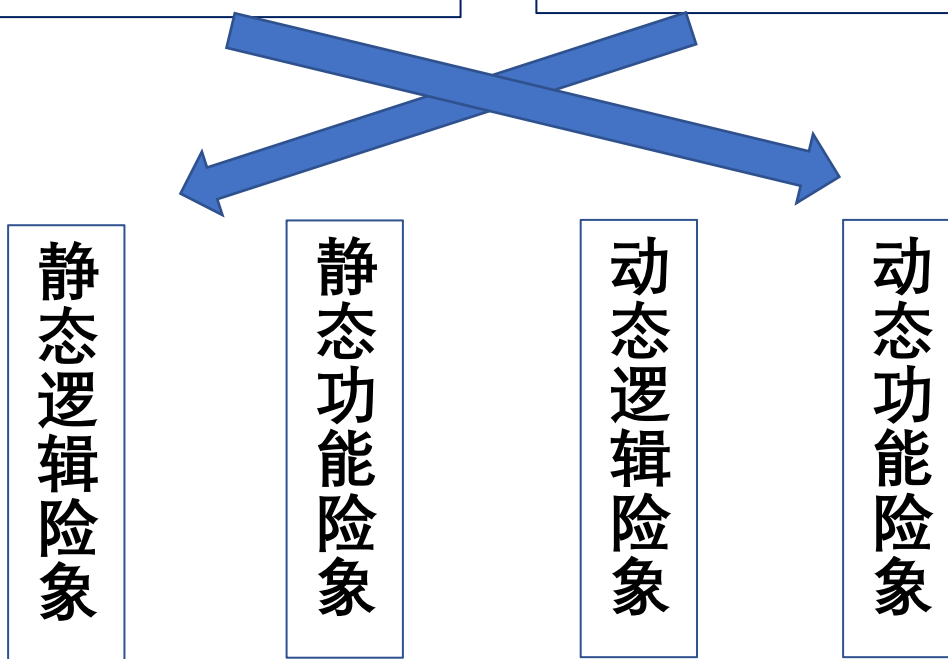
4.5.2 险象的分类

依据输入信号变化前后输出信号的变化情况，分为：

- **静态险象**：本应不变而发生了变化
- **动态险象**：本应一次变化而发生了多次变化

依据导致输出信号发生变化的输入变量个数，分为：

- **逻辑险象**：一个输入变量发生变化导致的险象
- **功能险象**：多个输入变量发生变化导致的险象



4.5.2 险象的分类

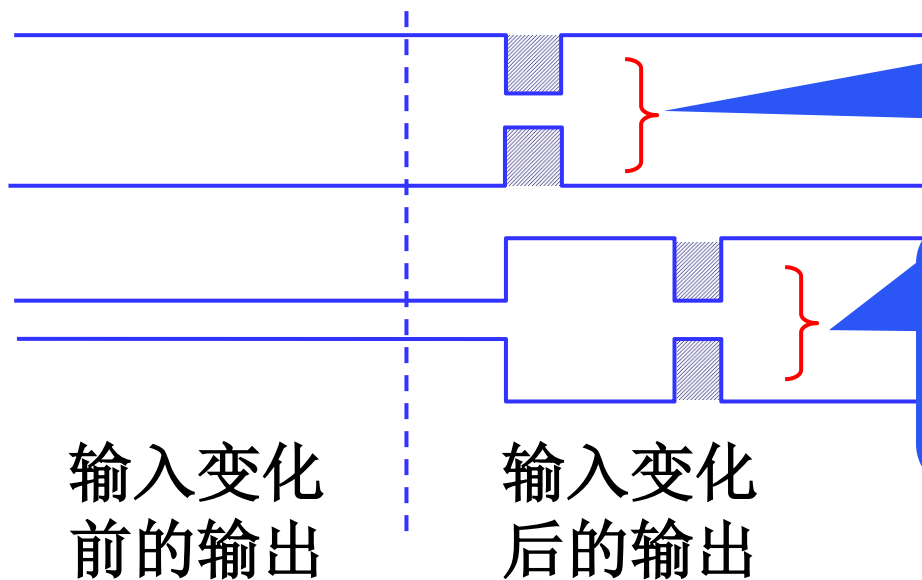
输出错误 { 0型险象：产生低电平错误
1型险象：产生高电平错误

静态0型

静态1型

动态0型

动态1型



本应不变而
发生了变化

本应一次变
化而发生了
多次变化

一、静态功能险象

◆ 原因：

多个输入变量的值不可能严格地“同时”变化

◆ 条件：

- K 个 ($K > 1$) 输入信号同时发生变化
- 输入信号变化前、后的稳态输出值相同
- 变化的 K 个变量的取值组合，对应在卡诺图上所占有的 2^K 个方格中，必定既有1，又有0

4.5.2 险象的分类

一、静态功能险象

【例4.5.1】分析逻辑函数 $F = B\bar{C} + AC$ ，说明当输入信号ABC由010变化到111时，是否有险象发生

分析：

输入信号A和C发生变化，从00变为11；B=1

AB \ C		00	01	11	10
C	0	0	1	1	0
	1	0	0	1	1

B=1，A和C发生变化的4个最小项中既有1，也有0



电路有可能发生险象

4.5.2 险象的分类

一、静态功能险象

$$F = \overline{B}\overline{C} + AC$$

AB		00	01	11	10
C	0	0	1	1	0
	1	0	0	1	1

AC同时从00→11，有两条变化路径

路径1: A先变，C再变

组合变化顺序: 010→110→111

输出F: 1→1→1

无险象发生

路径2: C先变，A再变

组合变化顺序: 010→011→111

输出F: 1→0→1

险象发生

功能险象是逻辑函数的功能所固有的，无法通过改变设计来消除，只能通过控制输入信号的变化顺序来避免

二、静态逻辑险象

原因：

逻辑器件固有的时延

条件：

- 一对逻辑变量A和反变量 $\bar{A}$ 同时出现，且在某些取值条件下，逻辑表达式可写成 $F = A \cdot \bar{A}$ 或者 $F = A + \bar{A}$
- 输入变量A发生变化
- 输入变量发生变化前、后的稳态输出值相同

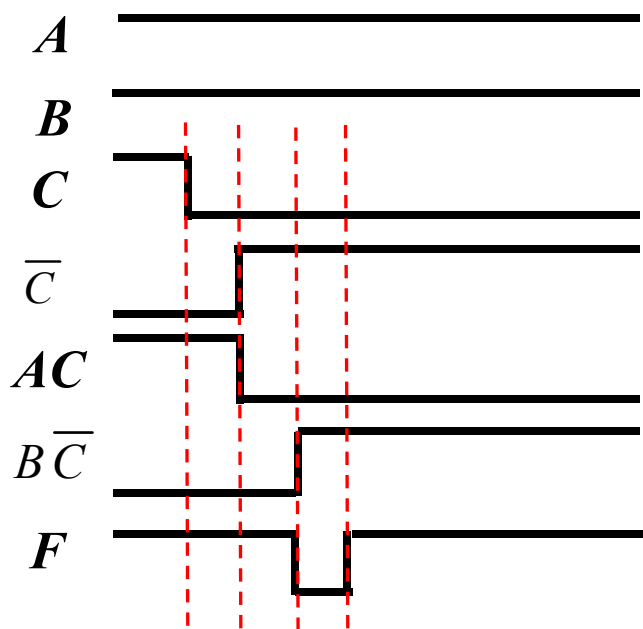
4.5.2 险象的分类

二、静态逻辑险象

$$F = B\bar{C} + AC$$

当A=B=1时, $F = C + \bar{C}$,

当C由1变为0, 会出现险象



AB \ C		00	01	11	10
C	0	0	1	1	0
	1	0	0	1	1

A=B=1, C变化,
对应在卡诺图上
2个方格全为1或
全为0, 险象发生

4.5.3 险象的判别

一、逻辑表达式判别法

如果电路中存在出现险象的可能性，则其逻辑表达式有如下特点：

- 当**某一变量同时以原变量和反变量的形式出现在逻辑表达式中**，则该变量就具备了竞争的条件
- 保留被研究变量，用某些定值消去其它变量
- 若得到的表达式为下列形式之一，则有险象存在

$$F = A + \bar{A}$$

0 险象（如**A**从**1** → **0**）

$$F = A \cdot \bar{A}$$

1 险象（如**A**从**0** → **1**）

4.5.3 险象的判别

一、逻辑表达式判别法

例如： $F = (A + B + \bar{C})(C + D)(\bar{B} + \bar{D})$

分析：式中变量**B、C、D**均以原变量、反变量形式出现在表达式中，具备竞争条件。

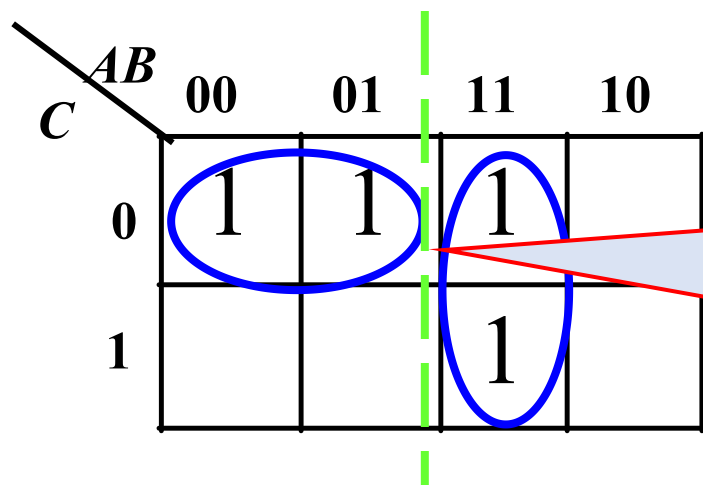
- 当ACD=011时，表达式为 $F = B \cdot \bar{B}$ ，如果B从0→1，则存在1险象。
- 当ABD=000时，表达式为 $F = C \cdot \bar{C}$ ，如果C从0→1，则存在1险象。
- 当ABC=010或110时，表达式为 $F = D \cdot \bar{D}$ ，如果D从0→1，则存在1险象。

4.5.3 险象的判别

二、卡诺图判别法

对于“**与-或**”电路：在卡诺图中，如果两个圈“1”的卡诺圈存在着部分**相切**，且这个相切部分又没有被其它的圈“1”卡诺圈包含，则该电路必然存在险象

$$F = \overline{A}\overline{C} + AB$$



相切处是某个变量的交替界面，就是该变量由1→0，引起险象的发生

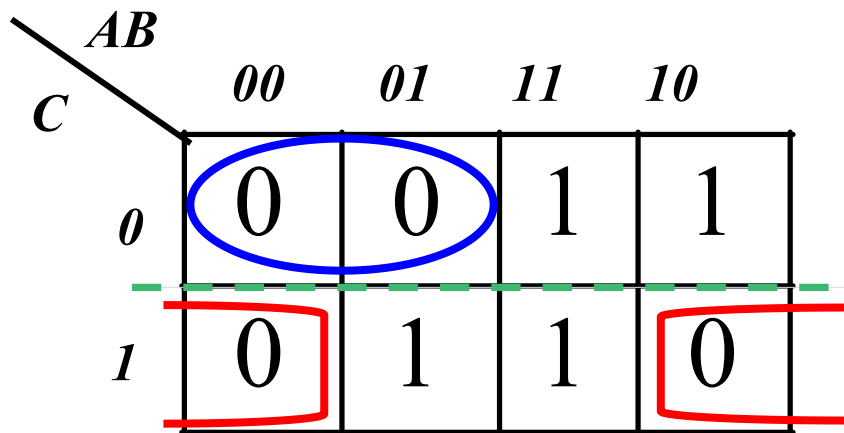
本例为变量A

4.5.3 险象的判别

二、卡诺图判别法

对于“或-与”电路：在卡诺图中，如果两个圈“0”的卡诺圈存在着部分相切，且这个相切部分又没有被其它的圈“0”卡诺圈包含，则该电路必然存在险象

$$F = (A + C)(B + \bar{C})$$

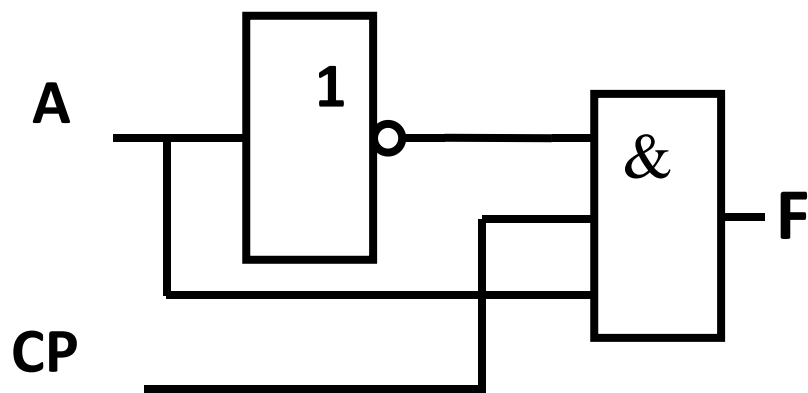


相切处是某个变量的交替界面，就是该变量由0→1，引起险象的发生

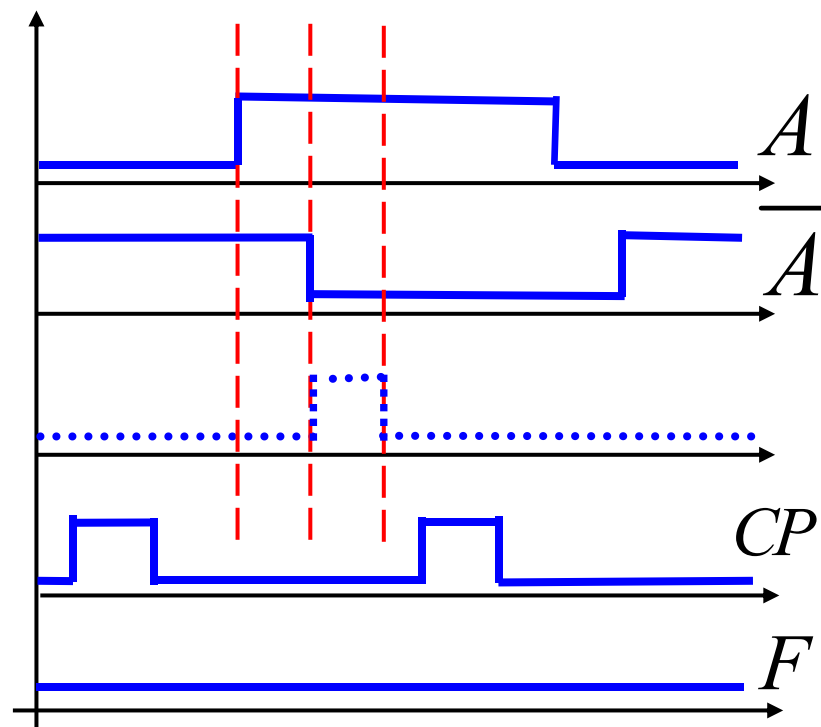
本例为变量C

4.5.4 险象的消除

一、加选通脉冲



- (1) 先使 $CP=0$ ，关闭与门
- (2) 等 A 、 $\bar{A}$ 信号都来到后，让 $CP=1$ ，得到可靠的 F



4.5.4 险象的消除

二、加冗余卡诺圈

$$F = A\bar{B} + BC$$

AB		00	01	11	10
C	0	0	0	0	1
	1	0	1	1	1

$$F = A\bar{B} + BC + AC$$

$$F = (A + B)(\bar{B} + C)$$

AB		00	01	11	10
C	0	0	0	0	1
	1	0	1	1	1

$$F = (A + B)(\bar{B} + C)(A + C)$$

第四章习题

#19 用数据选择器74LS151实现下列逻辑函数。

$$(1) f(A_1, B_2, C_3) = \sum m(0, 2, 3, 4, 5, 7)$$

$$(2) f(A_1, B_2, C_3) = \sum m(1, 2, 3, 5, 6)$$

#22 试用1片3线-8线译码器74LS138构成1位全减器。

#27 试判断逻辑函数 $F = \overline{A}\overline{B}C + \overline{A}B\overline{C} + BD$ 构成的组合逻辑电路是否存在竞争和险象。若存在，说明是哪一种类型，可以采取哪些措施避免险象的发生。



武汉大学
Wuhan University

谢谢!

2025-11-20

